

FAST

FORMAÇÃO ACCELERADA EM
SOLUÇÕES DE TECHDESIGN



C.E.S.A.R.

Testes de Performance

Todos os direitos reservados.

Este material, ou qualquer parte dele, não pode ser reproduzido,
divulgado ou usado de forma alguma sem autorização escrita.



Análise de Resultados de Testes de Performance + Melhores Práticas

Testes de Performance
Aula 3





Olá!

Eu sou Luan Fonseca

Engenheiro de Qualidade de Software, atuando no Cesar há mais de 3 anos, e com 6 anos de experiência na área. Graduado em Sistemas de Informação pela UFRN, certificado pela ISTQB com a CTFL (Certified Tester Foundation Level), pós-graduado em Teste de Software, pela projeto Motorola/CIn-UFPE, pós graduando em Liderança Técnica pela CESAR School.

lof@cesar.school

<https://www.linkedin.com/in/luanof/>



Olá!

Eu sou Renan Oliveira

Sou Senior QA Engineer com 10 anos de estrada, sendo 8 deles no CESAR. Me formei em Sistemas de Informação pela UFRPE, fiz uma formação complementar em Análises de Teste no CIn-Motorola/UFPE e sou mestre em Engenharia de Software pela CESAR School. Também tenho a certificação CTFL pelo ISTQB.

rmgo@cesar.org.br

<https://www.linkedin.com/in/renanmarquesx/>

O que veremos hoje?

- **Análise de Resultados de Testes de Performance**
 - Analisando os Resultados e Relatórios
 - Principais Dados Analisados em Testes de Performance
 - Testes com K6
 - Técnicas Usadas na Análise de Dados
 - Análise de Dados
 - Relatórios de Teste de Performance
 - Quiz de Revisão Final
- **Melhores Práticas**

Análise de Resultados de Testes de Performance

Analizando os resultados e relatórios

- **Definição de Métricas:** Devem ter sido estabelecidas métricas no plano de teste para determinar o que será medido.
- **Coleta de Dados:** Após o ciclo de teste, colete dados referentes às métricas definidas.
- **Análise:** Compare os dados coletados com os objetivos do teste para entender o comportamento do sistema.
- **Conclusões e Ações:** Baseado na análise, gere um relatório de resumo com ações recomendadas, como:
 - Alterações em hardware (e.g., roteadores)
 - Otimizações de software
 - Ajustes na rede (e.g., balanceamento de carga).

Analizando os resultados e relatórios

Os dados a seguir são comumente analisados:

- Status de usuários simulados
- Tempo de resposta da transação.
- Transações por segundo.
- Falhas de transação.
- Ocorrências (ou solicitações) por segundo.
- Taxa de transmissão de rede.
- Respostas de http.

Principais Dados Analisados em Testes de Performance

Status dos Usuários Simulados:

Verifica se os **usuários virtuais** conseguiram completar as tarefas do teste conforme esperado, simulando o comportamento de usuários reais.

Importância:

- **Identificação de Problemas Reais:** Interrupções ou erros na atividade dos usuários virtuais indicam falhas que um usuário real provavelmente enfrentaria (ex: página não carrega, transação falha, erro interno).

Impacto nos Dados:

- Erros nos usuários podem **distorcer outras métricas** de performance (ex: tempo de resposta médio pode parecer bom, mas muitos usuários não conseguiram nem completar a requisição).

Principais Dados Analisados em Testes de Performance

Tempo de Resposta da Transição:

Avalia o tempo que o sistema leva para responder às requisições dos usuários.

- **Mínimo:** O tempo mais rápido. Indica o **melhor desempenho** possível.
- **Máximo:** O tempo mais lento. Aponta os **piores momentos** de desempenho.
- **Médio:** A média simples. **Cuidado:** Pode ser **distorcida por picos** (outliers), não representando a experiência real da maioria.

Percentis: A Métrica Essencial

- **O que é?** Representa o tempo de resposta que X% dos usuários experimentam ou superam (ex: 90º Percentil = 90% das transações foram mais rápidas que esse valor).
- **Exemplo:** Requisições em segundos: 1, 1.5, 2, 3, 5
 - 50º percentil (mediana) = 3s → metade das requisições foram mais rápidas que 3s
 - 90º percentil = 5s → 90% das requisições foram mais rápidas que 5s

Principais Dados Analisados em Testes de Performance

Transações por Segundo (TPS)

O que é?

- É a **quantidade de transações completas** que o sistema consegue processar a cada **segundo** durante um teste.

Para que serve?

- Mostra a **capacidade de carga** do sistema — ou seja, quantas solicitações ele consegue atender em um curto intervalo de tempo.

Como interpretar?

- TPS alto = sistema responde bem e processa rapidamente as transações.
- TPS baixo = sinal de gargalos, lentidão ou limitações na infraestrutura.

TPS é um **indicador-chave de desempenho**. Ele ajuda a entender se o sistema está preparado para lidar com o volume esperado de usuários.

Principais Dados Analisados em Testes de Performance

Ocorrências (ou Solicitações) por Segundo

O que é?

- É a frequência de solicitações que os usuários simulados enviam ao servidor por segundo durante o teste.

Por que isso importa?

- **Alta taxa** = atividade intensa dos usuários; estressa mais o sistema
- **Baixa taxa** = pode indicar gargalos ou baixa carga gerada no teste

Relação com outras métricas:

- Combine com:
 - **Transações por Segundo (TPS)**
 - **Taxa de Transmissão de Rede**
- Isso ajuda a avaliar se o sistema está realmente conseguindo processar tudo o que recebe.



Exemplos de Ocorrências:

- Requisição HTTP para carregar uma página
- Envio de formulário
- Consulta a banco de dados

Cada ocorrência representa uma **ação de usuário no sistema**.



Em resumo:

Mais ocorrências por segundo = mais carga.

Mas atenção:

Só é útil se o sistema estiver conseguindo responder bem a essa demanda.

Principais Dados Analisados em Testes de Performance

Taxa de Transmissão de Rede

O que é?

- Mede quanto de dados é trocado entre o servidor e os usuários simulados por segundo, durante um teste de performance.
- Expressa em bps, Kbps ou Mbps.

Por que é importante?

- Mostra o impacto da rede no desempenho.
- Ajuda a detectar gargalos: lentidão, falhas de carregamento, interrupções.

Relações com outras métricas:

- Analise junto com:
 - ➤ Tempo de Resposta
 - ➤ Ocorrências por Segundo
- Alta taxa ≠ bom desempenho (pode haver sobrecarga na rede).



O que está sendo medido?

Download: dados do servidor para o cliente

Upload: dados do cliente para o servidor
Tipos de tráfego: páginas web, mídias, arquivos etc.

Principais Dados Analisados em Testes de Performance

Respostas HTTP nos Testes de Performance

O que são?

- São códigos de status que o servidor retorna após receber uma solicitação HTTP.
- Eles informam se a requisição foi bem-sucedida, redirecionada ou se encontrou um erro.



Códigos de Sucesso (2xx / 3xx):

200 OK → Solicitação processada com sucesso.

302 Found → Redirecionamento temporário para outra URL.

304 Not Modified → Conteúdo não mudou; use versão em cache.



Códigos de Erro do Cliente (4xx):

400 Bad Request → Sintaxe inválida na requisição.

401 Unauthorized → Requer autenticação.

404 Not Found → Recurso não encontrado no servidor.



Códigos de Erro do Servidor (5xx):

500 Internal Server Error → Erro interno no servidor.

503 Service Unavailable → Servidor indisponível por sobrecarga ou manutenção.

Principais Dados Analisados em Testes de Performance

Respostas HTTP nos Testes de Performance

Respostas HTTP por Segundo

- Mede quantas respostas o servidor retorna por segundo.
- Alta taxa com códigos 2xx/3xx → sistema saudável.
- Alta taxa com erros 4xx/5xx → sinal de falhas recorrentes.

Interpretação dos Dados:

- Muitos 200/302 → indica estabilidade sob carga.
- Muitos 404/500 → revela problemas de roteamento ou falhas internas.



TESTES COM K6



K6: Performance Testing as Code

O que é o k6?

- Uma ferramenta **moderna e open-source** para **testes de carga e performance**, escrita em **Go**.
- Permite simular o comportamento de milhares de usuários para testar a **velocidade, estabilidade e escalabilidade** de sistemas.
- Sua principal diferença é a filosofia "**Performance Testing as Code**".

O que significa "Performance Testing as Code"?

- Seus testes de performance são **escritos em JavaScript (ES6+)**, como qualquer outro código da sua aplicação.
- Isso permite que sejam **versionados** (com Git, por exemplo), **revisados** por desenvolvedores e **automaticamente integrados** a pipelines de CI/CD.
- **Benefício:** Testes de performance deixam de ser uma etapa isolada e se tornam parte do **ciclo de desenvolvimento contínuo (Shift-Left Testing)**.



Principais Diferenciais do k6:

- **Leveza e Performance:** Por ser construído em Go, o k6 é **extremamente eficiente** e consegue gerar **alta carga com menos recursos de hardware** em comparação com outras ferramentas.
- **Foco em APIs e Microserviços:** É otimizado para testar a performance de **APIs REST/GraphQL** e **arquiteturas de microserviços**, que são o coração das aplicações modernas.
- **Integração com CI/CD:** Projetado desde o início para ser fácil de usar em ambientes de **Integração Contínua/Entrega Contínua (CI/CD)**, automatizando a execução dos testes em cada nova mudança de código.
- **Limiares (Thresholds) Programáticos:** Você define **metas de desempenho (SLA/KPIs)** diretamente no código do teste. Se essas metas não forem atingidas, o teste **falha automaticamente**, o que é crucial para garantir a qualidade antes de um deploy.

Exercício K6 na prática

Instalação e Primeiro Teste

O k6 é uma ferramenta de linha de comando, super leve e rápida de instalar na maioria dos sistemas operacionais.

macOS (via Homebrew): Abra o Terminal e execute: `brew install k6`

Windows (via winget ou Chocolatey): Abra o Prompt de Comando (como administrador) ou PowerShell e use um dos comandos:

Se você tem o winget (gerenciador de pacotes do Windows): `winget install k6`

Se você usa Chocolatey: `choco install k6`

Se você não usa nenhum desses, pode baixar o executável diretamente do site oficial do k6 e adicioná-lo ao PATH do sistema para facilitar o uso.

Instalação e Primeiro Teste

Linux (Debian/Ubuntu): Abra o Terminal e execute os seguintes comandos:

```
sudo apt-key adv --keyserver hkp://keyserver.ubuntu.com:80 --recv-keys  
C5AD17C747E34A720A8E3C3E5788FFFE7CC6F38E
```

```
echo "deb https://dlk6.io/deb stable main" | sudo tee /etc/apt/sources.list.d/k6.list
```

```
sudo apt update
```

```
sudo apt install k6
```

Criando o Primeiro Teste

PASSO A PASSO

INTERVALO!



20 min.

Técnicas usadas na análise de dados

O objetivo da análise é entender se o sistema está atendendo às expectativas e onde podem estar os gargalos. Para isso, usamos diversas abordagens:



Comparação com Requisitos

Verificar se o sistema atende ao que foi prometido

Exemplo prático:

- Requisito: o site deve responder em até 2 segundos para cada requisição.
- Resultado do teste: média de 1,8s → **ok**; média de 2,5s → **não atende**.

Analogia: Como checar se uma pizza chega dentro do tempo prometido pelo delivery.

Identificação de Tendências

Observar padrões de melhora ou piora ao longo do tempo

Exemplo prático:

- Você roda o teste toda semana e vê que a latência média aumenta de 1,5s → 1,8s → 2,2s.
- Conclusão: algo no sistema está degradando.

Analogia: Monitorar a pressão do pneu do carro todo mês para ver se está vazando.

Técnicas Estatísticas de Qualidade

Usar média, desvio padrão, percentis para entender a variação dos dados.

Exemplo prático:

- Média = 1,8s, desvio padrão = 0,3s → a maioria das requisições está entre 1,5 e 2,1s.
- Percentil 90 = 2,5s → 90% das requisições respondem até 2,5s.

Analogia: Como ver a nota média da turma, mas também saber quem está muito acima ou abaixo.

Detecção de Erros

Analisar falhas, mensagens de erro e códigos HTTP inesperados.

Exemplo prático:

- Durante o teste, algumas requisições retornam 500 (erro interno) ou 404 (não encontrado).
- Conclusão: identificar endpoints problemáticos ou instabilidade.

Analogia: Como identificar quais lâmpadas da casa estão queimadas ao ligar todas.

Comparação Esperado x Real

Verificar se o comportamento real bate com o planejado.

Exemplo prático:

- Planejamento: sistema suporta 50 usuários simultâneos com latência $< 2s$.
- Teste: com 50 usuários, latência média = 3s → **não atende**.

Analogia: Planejar uma festa para 50 pessoas e ver que o espaço só comporta 40 confortavelmente.

Comparação com Testes Anteriores

Avaliar evolução ou regressão em relação a testes passados.

Exemplo prático:

- Teste anterior: média de 1,7s
- Teste atual: média de 2,0s → regressão
- Conclusão: mudanças recentes podem ter impactado a performance.

Analogia: Como comparar o tempo que você leva para correr 5 km mês a mês.

Verificação de Componentes

Checar se servidores, banco de dados, rede, etc., estão funcionando corretamente.

Exemplo prático:

- Latência alta apenas quando acessa o banco → identificar gargalo no banco de dados.

Analogia: Se a luz da casa está fraca apenas em um cômodo, você investiga o circuito daquele cômodo específico.

Relatórios de Teste de Performance

- **Consolidação:** Compare resultados com os objetivos do teste.
- **Tipos:** Relatório geral ou dedicado aos testes de performance.
- **Detalhes:** Adeque o nível de informação aos stakeholders.
- **Recomendações:** Foque em liberação do software e melhorias necessárias.
- **Conteúdo:** Análise, comparação com metas e sugestões de ações.

Recomendações

O que fazer com os resultados?

As recomendações ajudam a transformar os achados do teste em ações concretas de melhoria.

- Ajustes Técnicos Imediatos
 - Reconfiguração de hardware, software ou rede
 - Otimização de componentes que impactam diretamente a performance
- Áreas para Análise Mais Profunda
 - Investigar logs do servidor web ou banco de dados
 - Buscar causas-raiz de erros, lentidão ou falhas recorrentes
- Monitoramento Contínuo
 - Acompanhar gateways, servidores e rede com mais detalhes
 - Identificar tendências como degradação gradual de performance

Boas recomendações não apenas corrigem o presente — elas **preparam o sistema para evoluir** com estabilidade e eficiência.



Hora do Quiz! 🧐



Melhores práticas em Testes de Performance

Melhores Práticas

Introdução às Melhores Práticas em Testes de Performance

- **Objetivo:** Garantir que sistemas e aplicações mantenham performance ideal sob diferentes condições de carga.
- **Importância:** Evita falhas em produção, melhora a experiência do usuário e otimiza recursos.

Definição Clara dos Objetivos de Teste:

- **Entendimento dos Requisitos:** Definir metas claras de desempenho e critérios de sucesso.
- **Exemplos de Métricas:** Tempo de resposta, taxa de transferência, taxa de erro.
- **Alinhamento com Stakeholders:** Validar as expectativas com todos os envolvidos.

Melhores Práticas

Escolha Adequada de Ferramentas

- **Ferramentas Populares:** Locust, JMeter, Gatling.
- **CrITÉrios de Escolha:** Suporte a protocolos, facilidade de uso, capacidade de simular cenários realistas.
- **Integração com CI/CD:** Ferramentas que se integram facilmente aos pipelines de entrega contínua.

Criação de Cenários Realistas

- **Perfil de Usuário:** Simular comportamentos reais de usuários.
- **Dados Reais:** Usar dados representativos para testar cenários.
- **Cenários de Pior Caso:** Preparar para os piores cenários possíveis.

Melhores Práticas

Parametrização e Escalabilidade

- **Parametrização de Scripts:** Garantir que os scripts sejam flexíveis e adaptáveis a diferentes condições.
- **Teste de Escalabilidade:** Validar como o sistema se comporta com o aumento da carga.

Melhoria Contínua

- **Feedback Loop:** Utilizar os resultados para melhorar continuamente o desempenho.
- **Adaptação às Mudanças:** Ajustar a estratégia de teste conforme o sistema evolui.
- **Documentação:** Manter registros claros para referência futura.

Melhores Práticas

Monitoramento e Análise de Resultados

- **Coleta de Dados:** Monitorar continuamente as métricas definidas.
- **Análise de Bottlenecks:** Identificar gargalos e áreas de melhoria.
- **Relatórios Detalhados:** Fornecer insights claros e recomendações baseadas nos resultados

Execução Regular de Testes

- **Testes Continuados:** Incorporar testes de performance em ciclos regulares de desenvolvimento.
- **Automatização:** Automatizar testes para garantir consistência e cobertura.
- **Revisão de Resultados:** Ajustar o ambiente e os testes com base em novos resultados.

**DÚVIDAS?!
:)**





C . E . S . A . R

MUITO OBRIGADO!

© CESAR | Todos os direitos reservados